

Cortex — Retrieval / RRF / Reranker Update

Full change summary · [hybrid.py](#) · [reranker.py](#) · [eval scripts](#) · [API](#) · [tests](#) · 2026-08-01

Scope

Parameterizes hybrid RRF fusion, adds optional cross-encoder reranker, shared labeled eval queries (20), quantitative sensitivity analysis, retrieval unit tests, and API opt-in wiring.

Files touched / added

Status	Path	Role
UPDATED	retrieval/hybrid.py	Parameterized RRF + optional reranker hook
NEW	retrieval/reranker.py	CrossEncoderReranker (ms-marco MiniLM)
NEW	scripts/eval_queries.py	20 labeled appsec queries (shared)
NEW	scripts/rrf_sensitivity.py	Grid sweep: hit_rate, MRR, rank-overlap
UPDATED	scripts/tune_rrf.py	Imports shared eval_queries
UPDATED	api/main.py	CORTEX_ENABLE_RERANKER env opt-in
NEW	tests/test_retrieval.py	Mocked unit tests for RRF + reranker
UPDATED	README.md	Retrieval tuning & reranking section

HybridRetriever parameters & defaults

Parameter	Default	Meaning
rrf_k	20	RRF smoothing (lower = sharper top ranks). Classic lit uses 60.
dense_weight	1.0	Multiplier on dense rank contribution
sparse_weight	1.0	Multiplier on BM25 rank contribution
over_fetch	3	Candidates per side = n_results × over_fetch
reranker	None	Optional CrossEncoderReranker instance
rerank	None	None=auto, True=force, False=skip
rerank_candidate_pool	20	RRF candidates handed to cross-encoder

Result shape (additive, non-breaking)

```
{
  "text": "...",
  "metadata": {...},
  "distance": 0.31,           # original dense distance (or None)
  "rrf_score": 0.0482,       # fused RRF score (higher = better)
  "rerank_score": 4.12       # only present when reranking ran
}
```

retrieval/reranker.py — CrossEncoderReranker

Model: cross-encoder/ms-marco-MiniLM-L-6-v2 (~80MB, CPU-friendly). Lazy-loaded on first use. Scores (query, document) pairs jointly — more accurate than independent embeddings, applied only to the small RRF candidate pool.

```
reranker = CrossEncoderReranker()
ranked = reranker.rerank(query, candidates)
# each candidate gains "rerank_score"; list sorted descending
```

scripts/eval_queries.py — 20 labeled queries

Shared source of truth for tune_rrf.py and rrf_sensitivity.py. Coverage: access control / authz, injection family (SQLi, XSS, command, SSRF), CSRF/session, API/JWT/rate-limit, LLM/prompt injection, Log4Shell CVEs, secure coding / secrets / misconfiguration.

Note: expected substrings must actually appear in your ingested corpus for hit_rate/MRR to mean anything.

scripts/rrf_sensitivity.py — quantitative sweep

Grid: $\text{rrf_k} \in \{5, 10, 20, 30, 60, 100\} \times \text{weights} \in \{\text{equal, dense-heavy, sparse-heavy}\}$. Metrics: hit_rate@k, MRR, rank-overlap (Jaccard of top-k sets vs previous k — low = sensitive). With `--with-reranker` also reports delta on average hit_rate/MRR.

```
python scripts/rrf_sensitivity.py
python scripts/rrf_sensitivity.py --with-reranker
```

api/main.py — opt-in reranker

```
# Off by default. Enable with:
export CORTEX_ENABLE_RERANKER=true
uvicorn api.main:app --reload
```

tests/test_retrieval.py + test status

Mocked unit tests (no chromadb/sentence-transformers required): rrf_score present, rerank=True without reranker raises, FakeReranker reorders, auto-use of configured reranker, Jaccard/MRR helpers. **14/14 tests passed** in this environment (9 security + 5 retrieval).

Compatibility

- api/main.py HybridRetriever(store, corpus_texts) picks up new defaults — no required edit.
- agent/orchestrator.py still reads results[0]["distance"] — field preserved.
- Result shape is additive only; existing callers ignoring extra keys keep working.

Suggested next steps

1. pip install -r requirements.txt
2. Drop real PDFs/DOCX into data/raw/, POST /ingest
3. python scripts/tune_rrf.py
4. python scripts/rrf_sensitivity.py --with-reranker
5. Promote winning (rrf_k, dense_weight, sparse_weight) into HybridRetriever defaults or api/main.py
6. Optionally set CORTEX_ENABLE_RERANKER=true once you decide the cross-encoder earns its cost